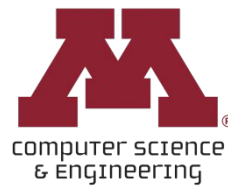


CSCI 5451: Introduction to Parallel Computing

Lecture 24: Convolutions in Cuda



UNIVERSITY OF MINNESOTA
Driven to Discover®

Announcements (12/01)

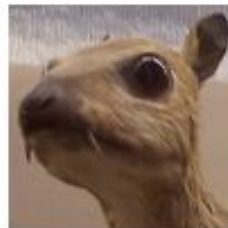
- ❑ Project responses given - be sure to check your group slack to ensure that you see expectations
- ❑ HW3 Due Yesterday
- ❑ HW4 Released (Due Dec 7)
 - Profiling a convolutional kernel
 - Done in Colab
- ❑ HW5 Released (Due Dec 18)
 - Group Assignment
 - Batch GEMM algorithm in CUDA



Convolutions

- ❑ Convolutional filters are arrays (1-d), matrices (2-d) and higher dimensional tensors (3-d) applied to input data
- ❑ These filters are also called kernels (we will use filters in later slides to avoid the confusion with cuda kernels)
- ❑ Have different uses
 - 1-D (Audio)
 - 2-D (Images)
 - 3-D (Video)

Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



Convolutions in 1-D

- ❑ Consider input data x of length n , filter f of length $2r + 1$
- ❑ r is often considered the *radius* of the convolution filter
- ❑ The output is some vector of data y

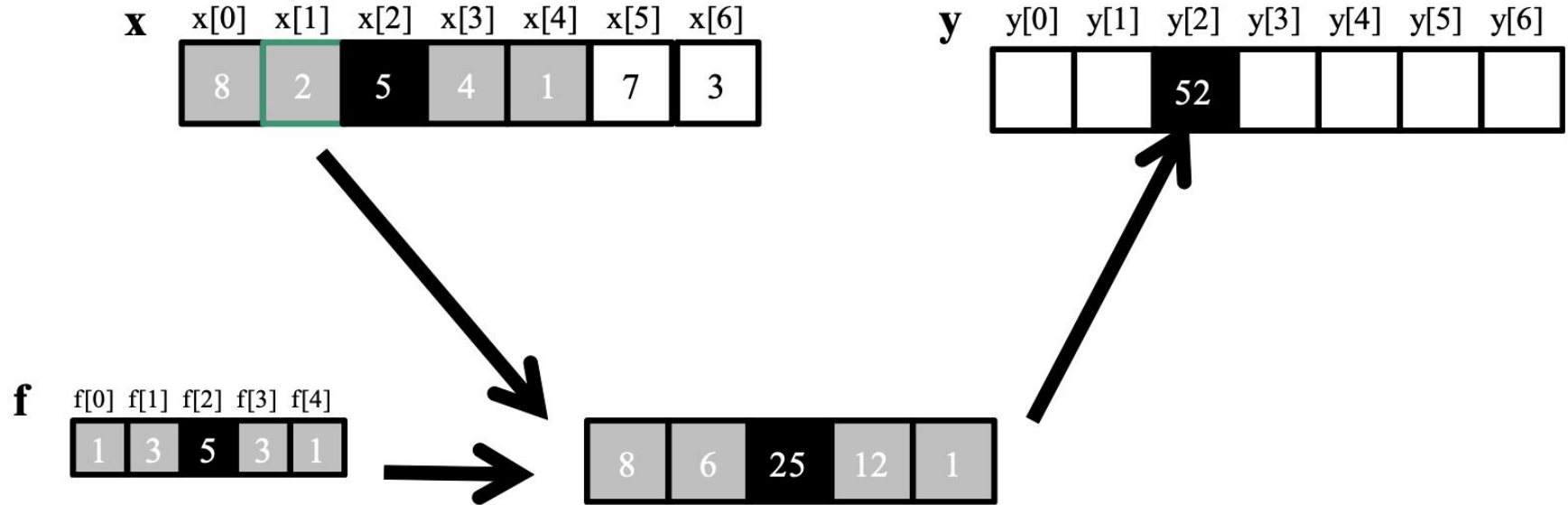
$$[x_0, x_1, \dots, x_{n-1}]$$

$$[f_0, f_1, \dots, f_{2r}]$$

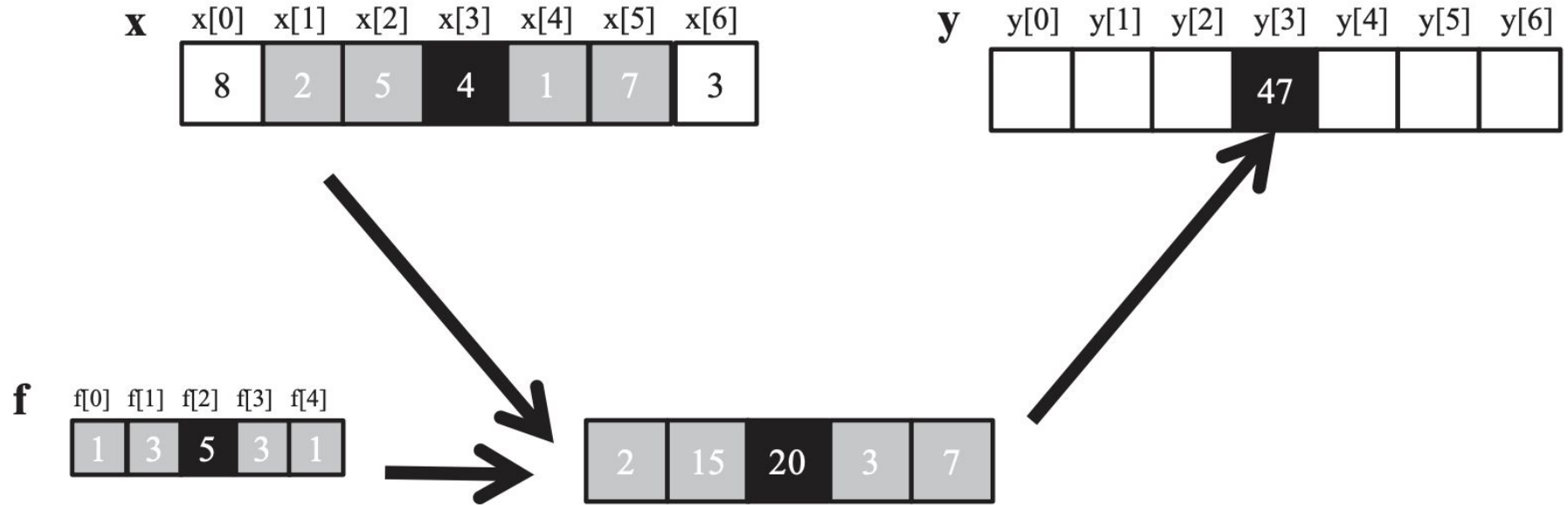
$$y_i = \sum_{j=-r}^r f_{i+j} \times x_{i+j}$$



1-D examples



1-D examples



How do we handle the data on the edge?

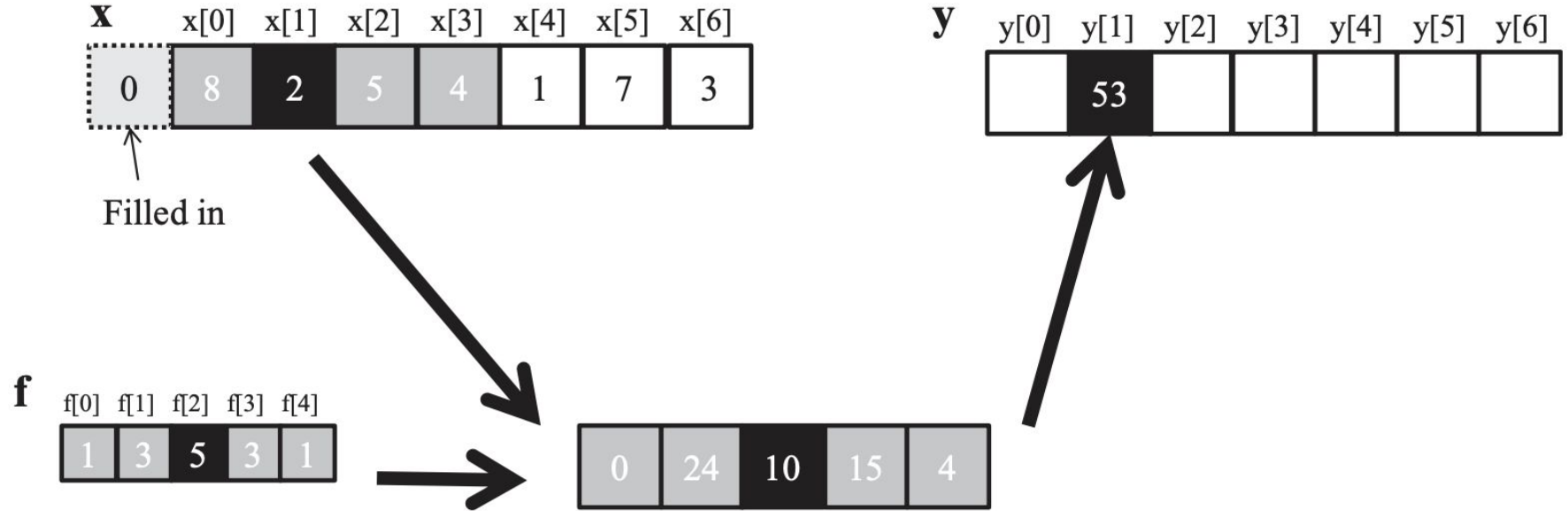


How do we handle the data on the edge?

Pad the inputs with
zeros.



1-D examples



2-D Convolution



$$P_{y,x} = \sum_{j=-r_y}^{r_y} \sum_{k=-r_x}^{r_x} f_{y+j,x+k} \times N_{y,x}$$



2-D Convolution

$$\begin{aligned}
 P_{2,2} &= N_{0,0} * M_{0,0} + N_{0,1} * M_{0,1} + N_{0,2} * M_{0,2} + N_{0,3} * M_{0,3} + N_{0,4} * M_{0,4} \\
 &\quad + N_{1,0} * M_{1,0} + N_{1,1} * M_{1,1} + N_{1,2} * M_{1,2} + N_{1,3} * M_{1,3} + N_{1,4} * M_{1,4} \\
 &\quad + N_{2,0} * M_{2,0} + N_{2,1} * M_{2,1} + N_{2,2} * M_{2,2} + N_{2,3} * M_{2,3} + N_{2,4} * M_{2,4} \\
 &\quad + N_{3,0} * M_{3,0} + N_{3,1} * M_{3,1} + N_{3,2} * M_{3,2} + N_{3,3} * M_{3,3} + N_{3,4} * M_{3,4} \\
 &\quad + N_{4,0} * M_{4,0} + N_{4,1} * M_{4,1} + N_{4,2} * M_{4,2} + N_{4,3} * M_{4,3} + N_{4,4} * M_{4,4} \\
 &= 1*1 + 2*2 + 3*3 + 4*2 + 5*1 \\
 &\quad + 2*2 + 3*3 + 4*4 + 5*3 + 6*2 \\
 &\quad + 3*3 + 4*4 + 5*5 + 6*4 + 7*3 \\
 &\quad + 4*2 + 5*3 + 6*4 + 7*3 + 8*2 \\
 &\quad + 5*1 + 6*2 + 7*3 + 8*2 + 5*1 \\
 &= 1 + 4 + 9 + 8 + 5 \\
 &\quad + 4 + 9 + 16 + 15 + 12 \\
 &\quad + 9 + 16 + 25 + 24 + 21 \\
 &\quad + 8 + 15 + 24 + 21 + 16 \\
 &\quad + 5 + 12 + 21 + 16 + 5 \\
 &= 321
 \end{aligned}$$

N

1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	5	6
5	6	7	8	5	6	7
6	7	8	9	0	1	2
7	8	9	0	1	2	3

P

		321				

f

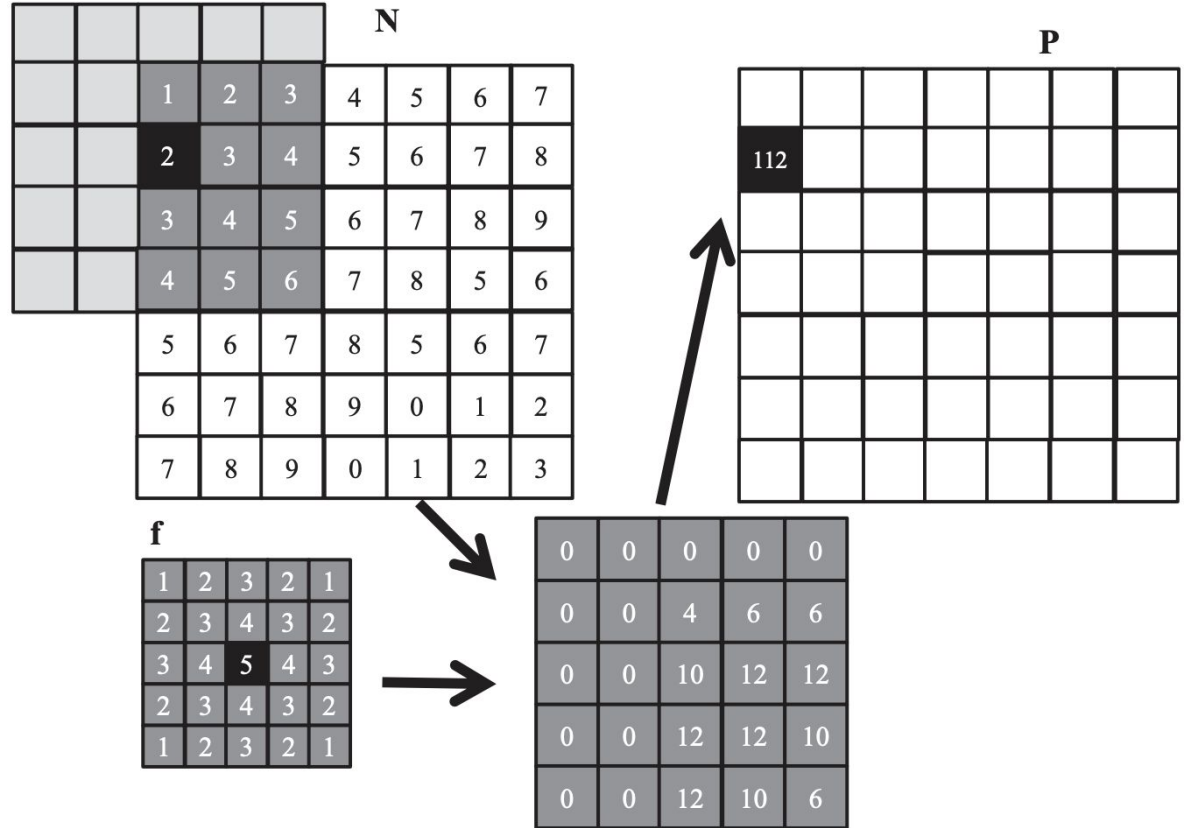
1	2	3	2	1
2	3	4	3	2
3	4	5	4	3
2	3	4	3	2
1	2	3	2	1

1	4	9	8	5
4	9	16	15	12
9	16	25	24	21
8	15	24	21	16
5	12	21	16	5



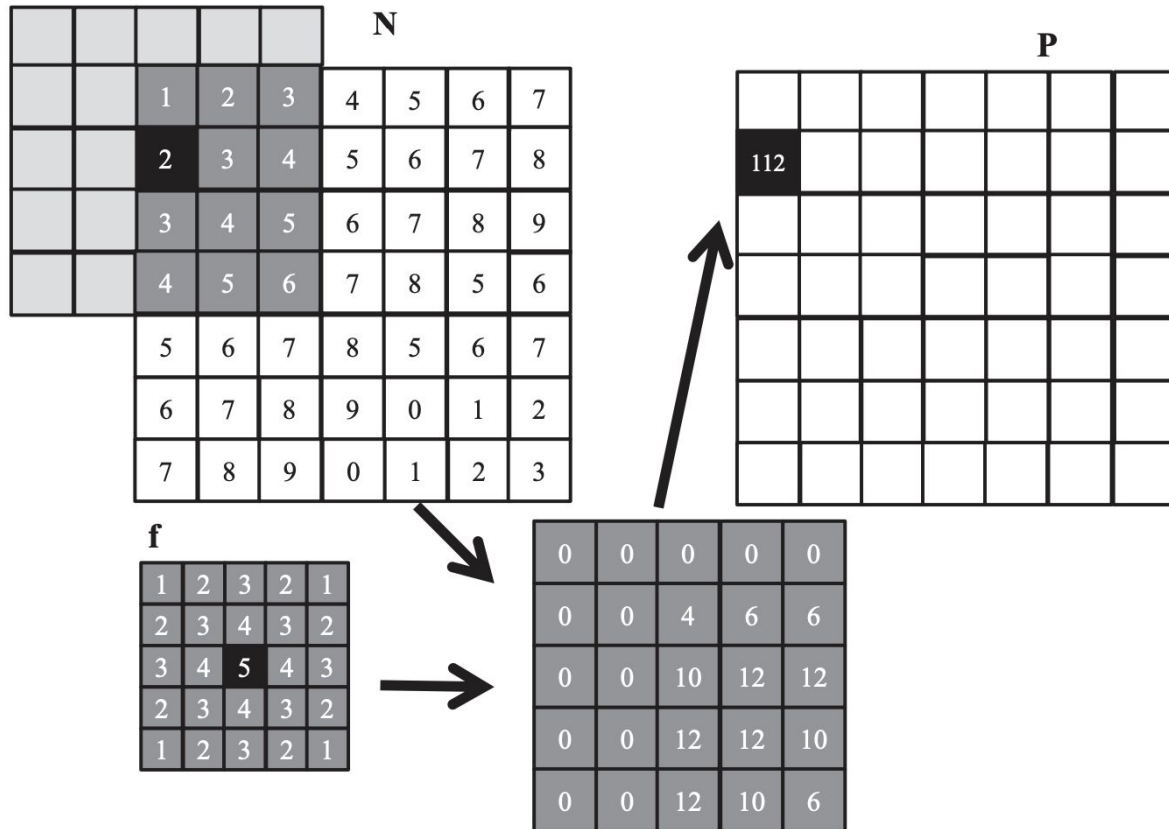
2-D Convolution

Similar to the 1-d case, we pad with zeros for the edge cases



2-D Convolution

How should we
parallelize this?



2-D Kernel without Constant Memory

```
01 __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,  
    int r, int width, int height) {  
02     int outCol = blockIdx.x*blockDim.x + threadIdx.x;  
03     int outRow = blockIdx.y*blockDim.y + threadIdx.y;  
04     float Pvalue = 0.0f;  
05     for (int fRow = 0; fRow < 2*r+1; fRow++) {  
06         for (int fCol = 0; fCol < 2*r+1; fCol++) {  
07             inRow = outRow - r + fRow;  
08             inCol = outCol - r + fCol;  
09             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {  
10                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];  
11             }  
12         }  
13     }  
14     P[outRow][outCol] = Pvalue;  
15 }
```



2-D Kernel without Constant Memory

Issues with this approach?

```
01 __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,  
    int r, int width, int height) {  
02     int outCol = blockIdx.x*blockDim.x + threadIdx.x;  
03     int outRow = blockIdx.y*blockDim.y + threadIdx.y;  
04     float Pvalue = 0.0f;  
05     for (int fRow = 0; fRow < 2*r+1; fRow++) {  
06         for (int fCol = 0; fCol < 2*r+1; fCol++) {  
07             inRow = outRow - r + fRow;  
08             inCol = outCol - r + fCol;  
09             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {  
10                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];  
11             }  
12         }  
13     }  
14     P[outRow][outCol] = Pvalue;  
15 }
```



2-D Kernel without Constant Memory

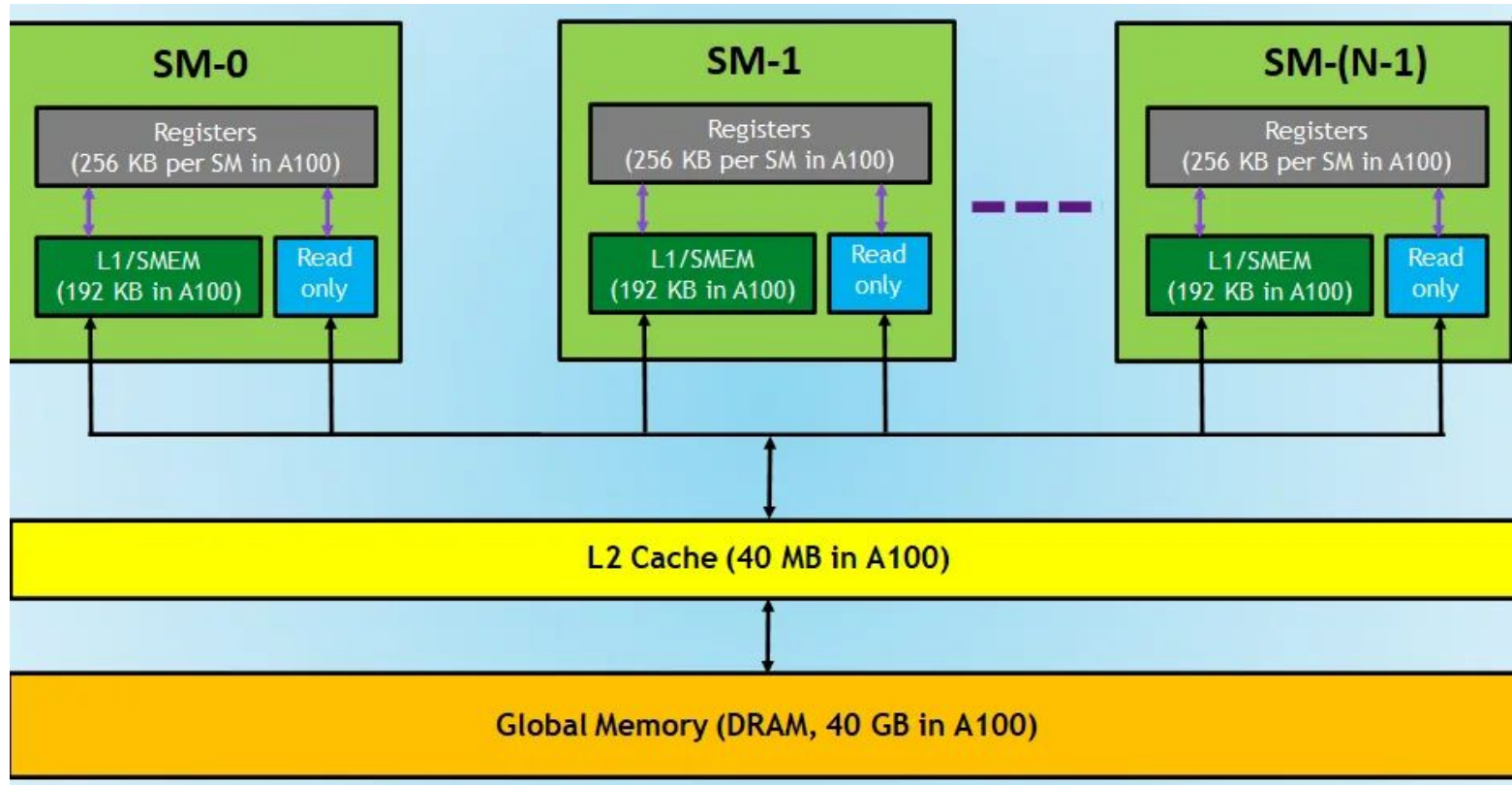
Issues with this approach?

Divergence, no constant memory, potentially low arithmetic intensity

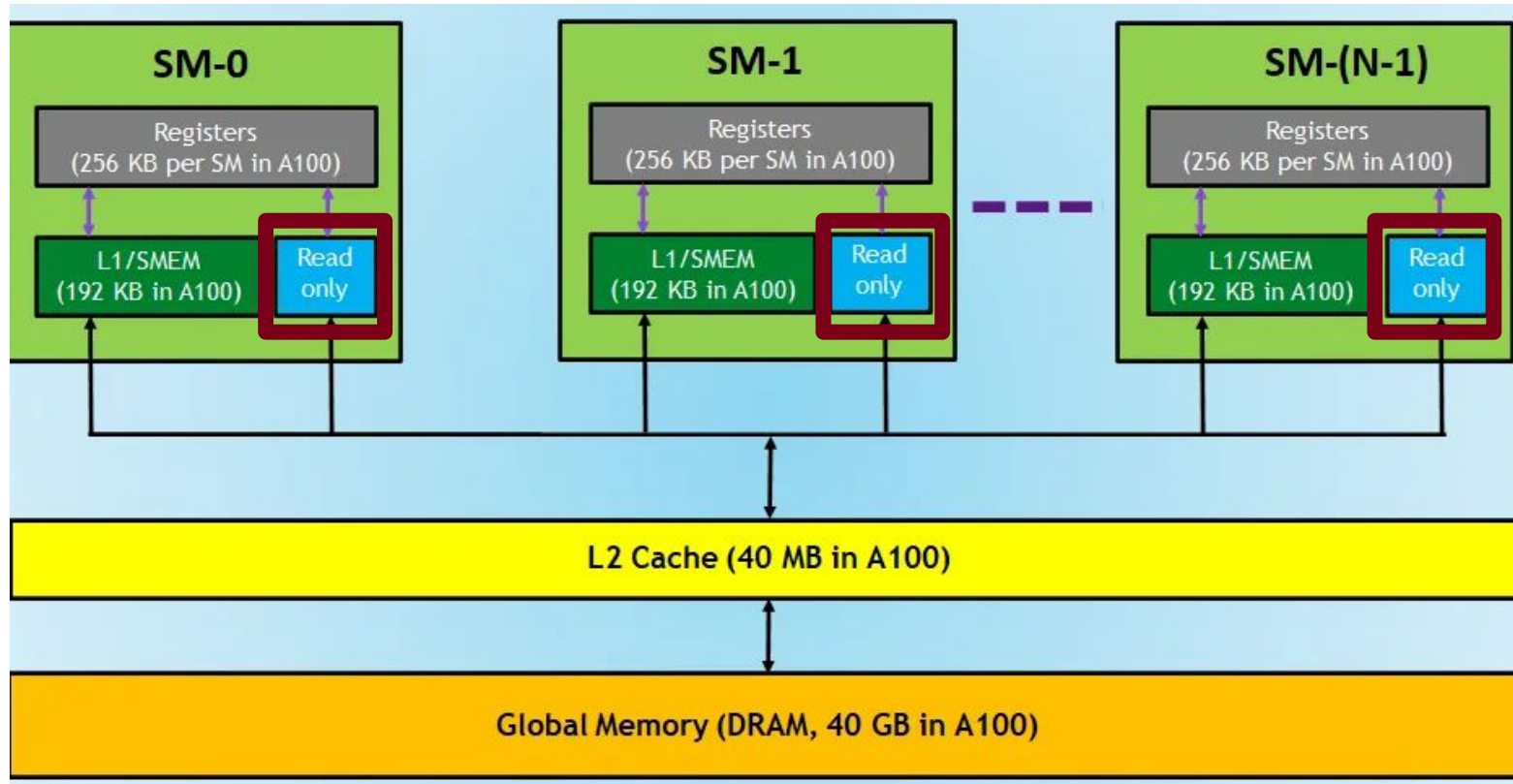
```
01 __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,  
    int r, int width, int height) {  
02     int outCol = blockIdx.x*blockDim.x + threadIdx.x;  
03     int outRow = blockIdx.y*blockDim.y + threadIdx.y;  
04     float Pvalue = 0.0f;  
05     for (int fRow = 0; fRow < 2*r+1; fRow++) {  
06         for (int fCol = 0; fCol < 2*r+1; fCol++) {  
07             inRow = outRow - r + fRow;  
08             inCol = outCol - r + fCol;  
09             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {  
10                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];  
11             }  
12         }  
13     }  
14     P[outRow][outCol] = Pvalue;  
15 }
```



Revisiting Constant Memory



Revisiting Constant Memory



Revisiting Constant Memory

- ❑ Constant memory is a read-only memory on each SM
- ❑ Allows for reduced logic in hardware as coherence is not necessary
- ❑ Access is faster than global and shared across all threads
- ❑ Useful for convolutions, where the filter is constant in execution

Variable declaration	Memory	Scope	Lifetime
Automatic variables other than arrays	Register	Thread	Grid
Automatic array variables	Local	Thread	Grid
<code>__device__ __shared__ int SharedVar;</code>	Shared	Block	Grid
<code>__device__ int GlobalVar;</code>	Global	Grid	Application
<code>__device__ __constant__ int ConstVar;</code>	Constant	Grid	Application



2-D Kernel with Constant Memory

```
#define FILTER_RADIUS 2
__constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];

__global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
int width, int height) {
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    float Pvalue = 0.0f;
    for (int fRow = 0; fRow < 2*r+1; fRow++) {
        for (int fCol = 0; fCol < 2*r+1; fCol++) {
            inRow = outRow - r + fRow;
            inCol = outCol - r + fCol;
            if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
            }
        }
    }
    P[outRow*width+outCol] = Pvalue;
}
```

```
cudaMemcpyToSymbol(F,F_h, (2*FILTER_RADIUS+1)*(2*FILTER_RADIUS+1)*sizeof(float));
```



2-D Kernel with Constant Memory

```
#define FILTER_RADIUS 2
__constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];

__global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
int width, int height) {
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    float Pvalue = 0.0f;
    for (int fRow = 0; fRow < 2*r+1; fRow++) {
        for (int fCol = 0; fCol < 2*r+1; fCol++) {
            inRow = outRow - r + fRow;
            inCol = outCol - r + fCol;
            if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
            }
        }
    }
    P[outRow*width+outCol] = Pvalue;
}
```

Same kernel
as before -
now using
constant F

```
cudaMemcpyToSymbol(F,F_h, (2*FILTER_RADIUS+1)*(2*FILTER_RADIUS+1)*sizeof(float));
```



2-D Kernel with Constant Memory

```
#define FILTER_RADIUS 2
__constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];

__global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
int width, int height) {
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    float Pvalue = 0.0f;
    for (int fRow = 0; fRow < 2*r+1; fRow++) {
        for (int fCol = 0; fCol < 2*r+1; fCol++) {
            inRow = outRow - r + fRow;
            inCol = outCol - r + fCol;
            if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
            }
        }
    }
    P[outRow*width+outCol] = Pvalue;
}
```

Same kernel
as before -
now using
constant F

This line inside of *main*

```
cudaMemcpyToSymbol(F,F_h, (2*FILTER_RADIUS+1)*(2*FILTER_RADIUS+1)*sizeof(float));
```



2-D Kernel with Constant Memory

How is memory loading from N working here?

```
#define FILTER_RADIUS 2
__constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];

__global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
int width, int height) {
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    float Pvalue = 0.0f;
    for (int fRow = 0; fRow < 2*r+1; fRow++) {
        for (int fCol = 0; fCol < 2*r+1; fCol++) {
            inRow = outRow - r + fRow;
            inCol = outCol - r + fCol;
            if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
            }
        }
    }
    P[outRow*width+outCol] = Pvalue;
}
```

Same kernel
as before -
now using
constant F

This line inside of *main*

```
cudaMemcpyToSymbol(F,F_h, (2*FILTER_RADIUS+1)*(2*FILTER_RADIUS+1)*sizeof(float));
```

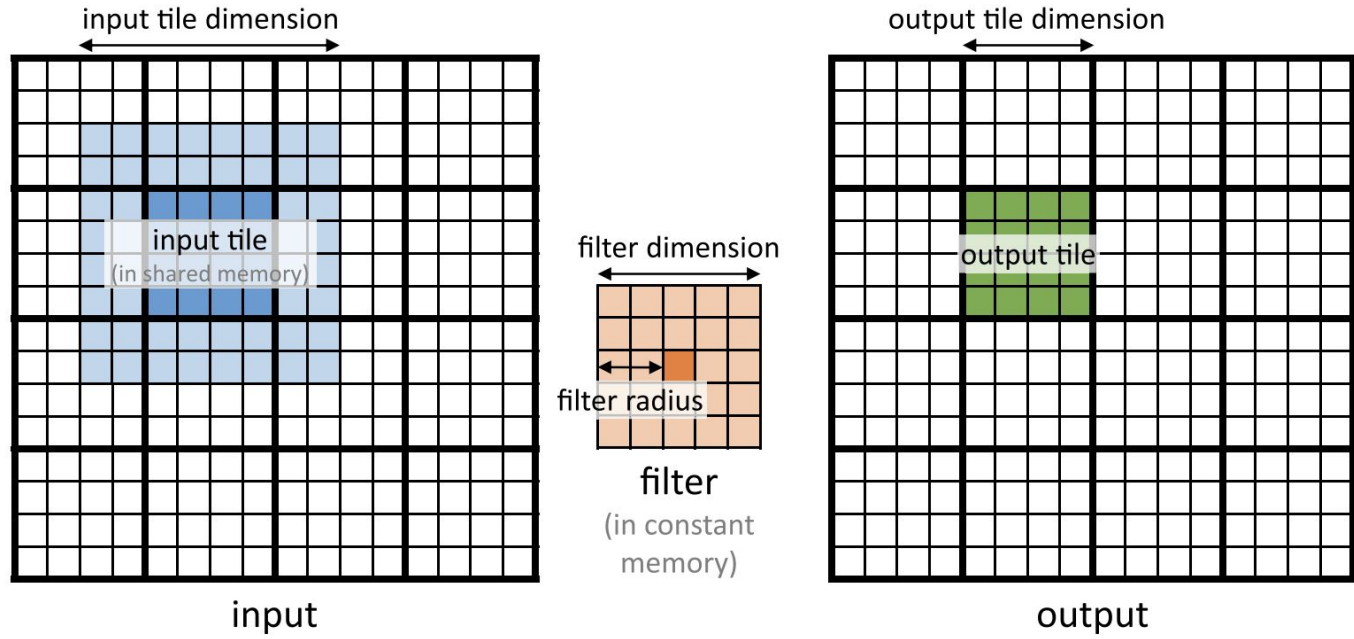


How else can we implement this?

Shared Memory Tiling

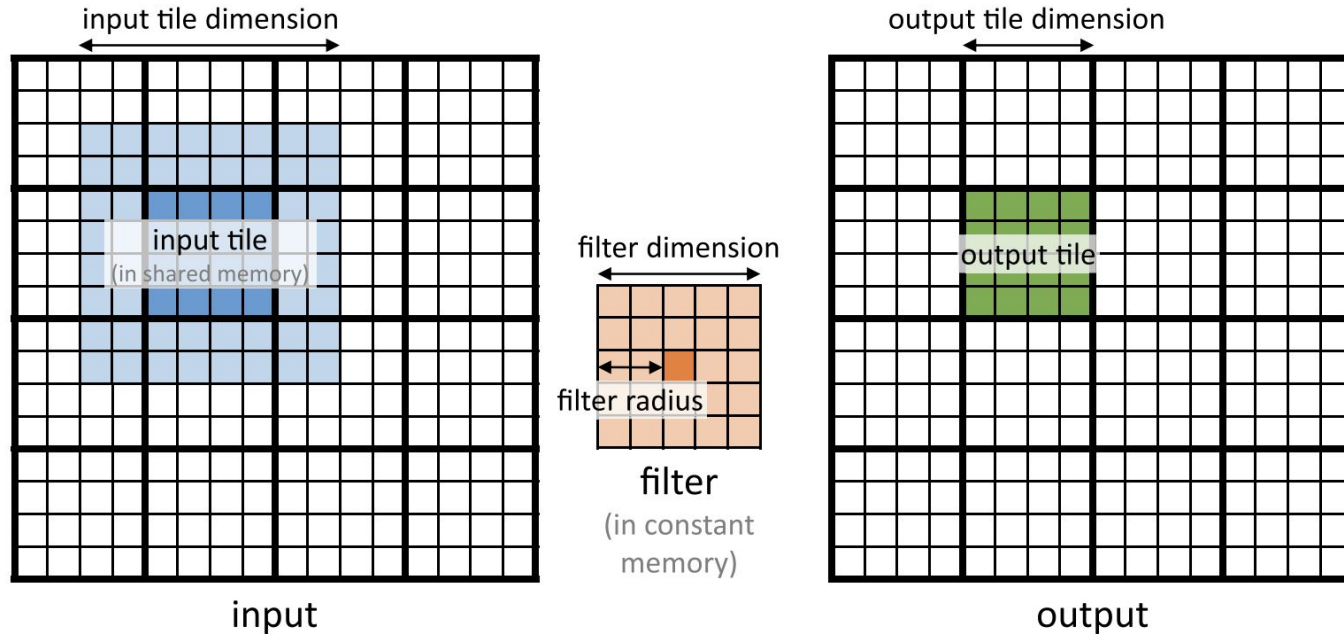


Tiled Kernel



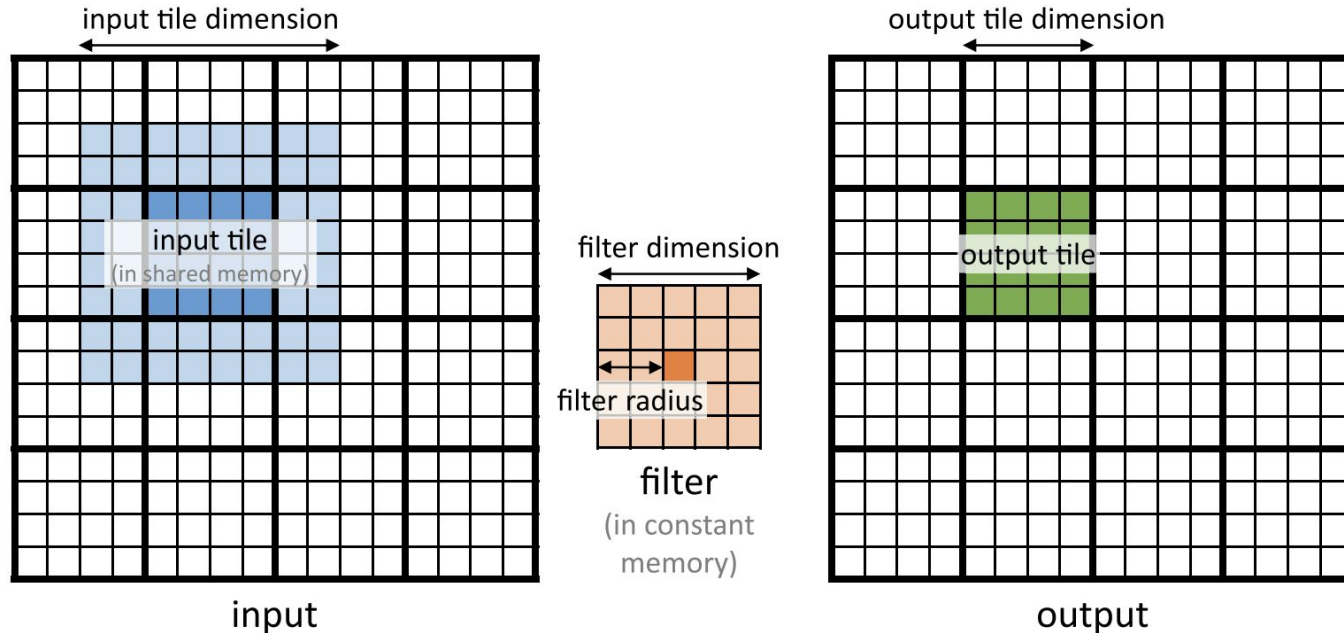
Tiled Kernel

Assume we launch 4x4 threadblocks.
(What is the problem with this size of threadblock?)



Tiled Kernel

We have to choose either to tile by input or by output. In other words, either every thread loads and only some compute (**input**) or every thread computes and loads more than once (**output**)



Tiled Kernel

```
01 #define IN_TILE_DIM 32
02 #define OUT_TILE_DIM ((IN_TILE_DIM) - 2*(FILTER_RADIUS))
03 __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
04 __global__ void convolution_tiled_2D_const_mem_kernel(float *N, float *P,
05                                                         int width, int height) {
06     int col = blockIdx.x*OUT_TILE_DIM + threadIdx.x - FILTER_RADIUS;
07     int row = blockIdx.y*OUT_TILE_DIM + threadIdx.y - FILTER_RADIUS;
08     //loading input tile
09     __shared__ N_s[IN_TILE_DIM][IN_TILE_DIM];
10     if(row>=0 && row<height && col>=0 && col<width) {
11         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
12     } else {
13         N_s[threadIdx.y][threadIdx.x] = 0.0;
14     }
15     __syncthreads();
16     // Calculating output elements
17     int tileCol = threadIdx.x - FILTER_RADIUS;
18     int tileRow = threadIdx.y - FILTER_RADIUS;
19     // turning off the threads at the edges of the block
20     if (col >= 0 && col < width && row >=0 && row < height) {
21         if (tileCol>=0 && tileCol<OUT_TILE_DIM && tileRow>=0
22             && tileRow<OUT_TILE_DIM){
23             float Pvalue = 0.0f;
24             for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
25                 for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
26                     Pvalue += F[fRow][fCol]*N_s[tileRow+fRow][tileCol+fCol];
27                 }
28             }
29             P[row*width+col] = Pvalue;
30         }
31     }
32 }
```



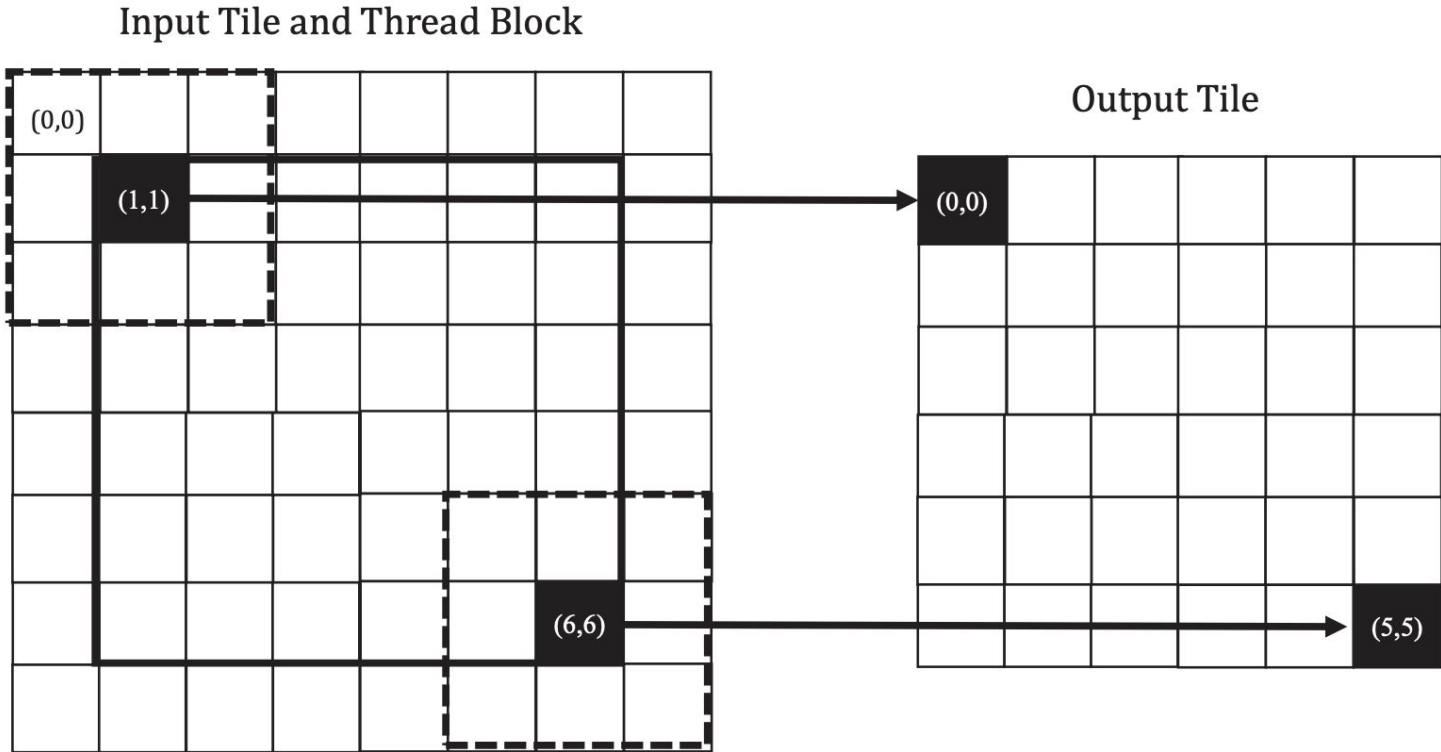
Tiled Kernel

All load, some
compute

```
01 #define IN_TILE_DIM 32
02 #define OUT_TILE_DIM ((IN_TILE_DIM) - 2*(FILTER_RADIUS))
03 __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
04 __global__ void convolution_tiled_2D_const_mem_kernel(float *N, float *P,
05                                                         int width, int height) {
06     int col = blockIdx.x*OUT_TILE_DIM + threadIdx.x - FILTER_RADIUS;
07     int row = blockIdx.y*OUT_TILE_DIM + threadIdx.y - FILTER_RADIUS;
08     //loading input tile
09     __shared__ N_s[IN_TILE_DIM][IN_TILE_DIM];
10     if(row>=0 && row<height && col>=0 && col<width) {
11         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
12     } else {
13         N_s[threadIdx.y][threadIdx.x] = 0.0;
14     }
15     __syncthreads();
16     // Calculating output elements
17     int tileCol = threadIdx.x - FILTER_RADIUS;
18     int tileRow = threadIdx.y - FILTER_RADIUS;
19     // turning off the threads at the edges of the block
20     if (col >= 0 && col < width && row >=0 && row < height) {
21         if (tileCol>=0 && tileCol<OUT_TILE_DIM && tileRow>=0
22             && tileRow<OUT_TILE_DIM){
23             float Pvalue = 0.0f;
24             for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
25                 for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
26                     Pvalue += F[fRow][fCol]*N_s[tileRow+fRow][tileCol+fCol];
27                 }
28             }
29             P[row*width+col] = Pvalue;
30         }
31     }
32 }
```



Tiled Kernel (previous slide) Approach



Alternative Tiled Kernel

We can also use a kernel which has every thread load in one entry, and compute one entry - then load all outside entries directly from global memory (this assumes we rely more heavily on L2 cache to store values across threadblocks)

```
01 #define TILE_DIM 32
02 __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
03 __global__ void convolution_cached_tiled_2D_const_mem_kernel(float *N,
04                                                             float *P, int width, int height) {
05     int col = blockIdx.x*TILE_DIM + threadIdx.x;
06     int row = blockIdx.y*TILE_DIM + threadIdx.y;
07     //loading input tile
08     __shared__ N_s[TILE_DIM][TILE_DIM];
09     if(row<height && col<width) {
10         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
11     } else {
12         N_s[threadIdx.y][threadIdx.x] = 0.0;
13     }
14     __syncthreads();
15     // Calculating output elements
16     // turning off the threads at the edges of the block
17     if (col < width && row < height) {
18         float Pvalue = 0.0f;
19         for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
20             for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
```

```
21                 if (threadIdx.x-FILTER_RADIUS+fCol >= 0 &&
22                     threadIdx.x-FILTER_RADIUS+fCol < TILE_DIM &&
23                     threadIdx.y-FILTER_RADIUS+fRow >= 0 &&
24                     threadIdx.y-FILTER_RADIUS+fRow < TILE_DIM) {
25                     Pvalue += F[fRow][fCol]*N_s[threadIdx.y+fRow][threadIdx.x+fCol];
26                 }
27             }
28         }
29         P[row*width+col] = Pvalue;
30     }
31 }
32 }
```



Alternative Tiled Kernel

We can also use a kernel which has every thread load in one entry, and compute one entry - then load all outside entries directly from global memory (this assumes we rely more heavily on L2 cache to store values across threadblocks)

```
01 #define TILE_DIM 32
02 __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
03 __global__ void convolution_cached_tiled_2D_const_mem_kernel(float *N,
04                                     float *P, int width, int height) {
05     int col = blockIdx.x*TILE_DIM + threadIdx.x;
06     int row = blockIdx.y*TILE_DIM + threadIdx.y;
07     //loading input tile
08     __shared__ N_s[TILE_DIM][TILE_DIM];
09     if(row<height && col<width) {
10         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
11     } else {
12         N_s[threadIdx.y][threadIdx.x] = 0.0;
13     }
14     __syncthreads();
15     // Calculating output elements
16     // turning off the threads at the edges of the block
17     if (col < width && row < height) {
18         float Pvalue = 0.0f;
19         for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
20             for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
```

```
21         if (threadIdx.x-FILTER_RADIUS+fCol >= 0 &&
22             threadIdx.x-FILTER_RADIUS+fCol < TILE_DIM &&
23             threadIdx.y-FILTER_RADIUS+fRow >= 0 &&
24             threadIdx.y-FILTER_RADIUS+fRow < TILE_DIM) {
25             Pvalue += F[fRow][fCol]*N_s[threadIdx.y+fRow][threadIdx.x+fCol];
26         }
27     } else {
28         if (row-FILTER_RADIUS+fRow >= 0 &&
29             row-FILTER_RADIUS+fRow < height &&
30             col-FILTER_RADIUS+fCol >= 0 &&
31             col-FILTER_RADIUS+fCol < width) {
32                 Pvalue += F[fRow][fCol]*
33                     N[(row-FILTER_RADIUS+fRow)*width+col-
34                     FILTER_RADIUS+fCol];
35             }
36         }
37     }
38     P[row*width+col] = Pvalue;
39 }
```



Connecting to HW4

- ❑ HW4 deals with all three of the discussed implementations using constant memory (+ 1 additional implementation)
- ❑ You will have to profile each of these & see what practical speedups look like
- ❑ So far, we have discussed that we should expect speedups when using shared memory → You have to verify whether this is true in practice for this kernel

